ASSIGNMENT 2: HEART FAILURE CLASSIFICATION PROBLEM

Abdel Rahman Mohamed Ahmed Ahmed Nasr
22010887

Lojine Sameh Abdel Moneam Abdel Gawad Morsy
22011054

Mohamed Sabry Mansour
22011129

April 14, 2026

ABSTRACT

1 Preprocessing

To prepare the raw data for analysis, a systematic preprocessing pipeline was developed to ensure the dataset was numerically compatible and statistically sound for various machine learning architectures.

1.1 Encoding and Partitioning

The initial phase involved transforming categorical features through **One-Hot Encoding**, expanding the feature space to capture distinct class categories. To ensure rigorous evaluation, the dataset was partitioned into three distinct sets using a stratified sampling approach:

- **Training Set (70%):** Utilized for the primary fitting of model weights and structures.
- **Validation (Dev) Set (10%):** Utilized as a proxy for unseen data to guide the tuning process and prevent overfitting.
- **Test Set (20%):** Held out exclusively for final reporting to ensure an unbiased measure of model performance.

1.2 Scaling and Dimensionality Reduction

Following the split, feature scaling was applied using a **StandardScaler**. This ensures that features with larger magnitudes do not disproportionately influence the model's loss function or the variance calculation in dimensionality reduction.

Principal Component Analysis (PCA) was employed to project the high-dimensional encoded data into a lower-dimensional subspace. Recognizing that different models have varying sensitivities to feature density and noise, the `n_components` parameter—representing the cumulative explained variance ratio—was not fixed. Instead, it was treated as a key hyperparameter.

For every algorithm implemented in this study, a **Hyperparameter Search** was conducted on the development set to determine the optimal variance threshold. This allowed each model to operate on a feature set that was uniquely optimized to balance information retention with noise reduction. All transformations (Scaling and PCA) were fitted on the training data and mapped onto the validation and test sets to maintain a strict firewall against data leakage.

2 Decision Tree

2.1 Approach

The Decision Tree was implemented using a recursive partitioning strategy. The model aims to maximize **Information Gain** (IG) by splitting the dataset at each node based on the feature that most effectively reduces entropy.

The entropy $H(S)$ of the dataset S is defined as:

$$H(S) = \sum_{i=1}^{c} -p_i \log_2 p_i \tag{1}$$

The Information Gain for a split on attribute A is then calculated as the difference between the original entropy and the weighted sum of the entropies of the resulting subsets:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \tag{2}$$

To determine the optimal configuration for the model, we utilized a **train-dev-test** split strategy. The development (validation) set was used to perform a grid search over various hyperparameters, ensuring that the model maintains a balance between bias and variance before final evaluation on the test set.

The best hyperparameters identified through this process were:

- **Max Depth:** 3
- **Min Samples Split:** 2
- **PCA n_components:** 0.85

2.2 Evaluation

The performance of the Decision Tree was assessed using the final held-out test set. By restricting the maximum depth to 3, the model remains highly interpretable while avoiding the risk of overfitting the training noise.

2.2.1 Metrics

The following performance metrics were obtained:

- **Accuracy:** 0.8478
- **F1-Score:** 0.8692

The high F1-Score relative to Accuracy suggests that the Information Gain criterion, combined with dimensionality reduction via PCA, successfully identified the most discriminative features for classification.

2.2.2 Confusion Matrix

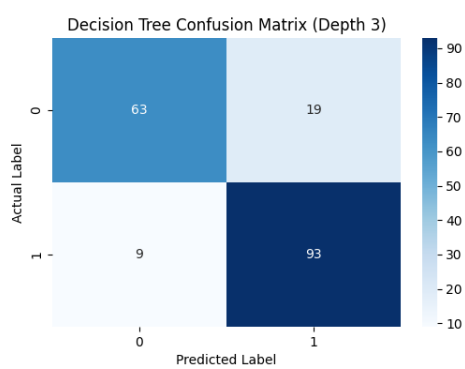

Figure 1: Confusion Matrix for Decision Tree with the Best Hyperparameters

3 Bagging Ensemble

3.1 Approach

Bagging (Bootstrap Aggregating) reduces model variance by training an ensemble of independent Decision Trees, each on a different bootstrap sample of the training data, and aggregating their predictions through a majority vote.

3.1.1 Bootstrap Sampling

For each of the M base learners, a bootstrap sample of size N is drawn with replacement from the training set. Approximately 63% of the original samples appear in each bootstrap, with the rest left out (out-of-bag samples). This diversity in training data decorrelates the individual trees, which is the key mechanism behind variance reduction.

3.1.2 Base Learner Training

Each base learner is a `DecisionTree` trained independently on its corresponding bootstrap sample. Unlike AdaBoost, no weighting or sequential dependency exists between learners—all trees are trained in parallel and contribute equally to the final prediction.

3.1.3 Majority Vote Aggregation

At inference time, every tree produces a class prediction for each sample. The final output is the class that receives the most votes across all M trees:

$$\hat{y} = \arg\max_{c} \sum_{m=1}^{M} \mathbb{I}(G_m(x) = c)$$

where $G_m(x)$ is the prediction of the m-th tree. This averaging process cancels out individual trees' errors, yielding a more stable and generalizable classifier.

The best hyperparameters identified through grid search on the validation set were:

- **Number of Estimators:** 20
- **Base Learner Max Depth:** 5
- **PCA n_components:** 0.9

3.2 Evaluation

3.2.1 Metrics

The following performance metrics were obtained on the held-out test set using the optimal hyperparameter configuration:

- **Accuracy:** 0.8641
- **F1-Score:** 0.8815

3.2.2 Confusion Matrix

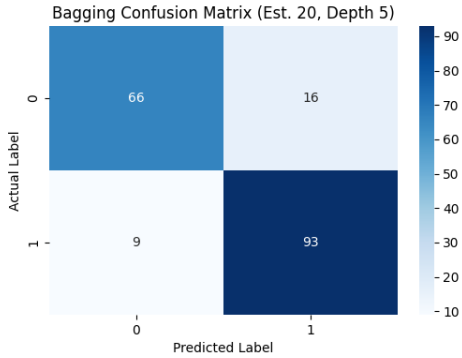

Figure 2: Confusion Matrix for Bagging Ensemble with the Best Hyperparameters

4 AdaBoost Ensemble

AdaBoost functions by training a sequence of weak learners—in this case, possible options were: stumps (1-level decision trees), shallow decision trees with a maximum depth of 2, and 3, where each subsequent model focuses specifically on the instances misclassified by its predecessor, unlike a standard tree that attempts to learn the entire data distribution at once.

4.1 Approach

4.1.1 Data Preparation and Label Mapping

While the base `DecisionTree` operates on standard $\{0, 1\}$ labels to satisfy the requirements of `np.bincount`, the AdaBoost mathematical framework requires labels $y \in \{-1, 1\}$. To accommodate this, the target vector was internally mapped to $\{-1, 1\}$ for weight updates and final voting, while the raw $\{0, 1\}$ labels were preserved during the training calls to the `DecisionTree.fit()` method.

4.1.2 Weight Initialization

The process begins by assigning an equal weight to every sample in the training set:

$$w_i = \frac{1}{N}$$

where N is the total number of training instances. This ensures that the initial weak learner treats all data points with equal importance.

4.1.3 Sequential Boosting Loop

For each of the M boosting rounds (where $M = 50$ was selected as the optimal hyperparameter), the following steps were performed:

- **Weighted Resampling:** Since the custom `DecisionTree` does not natively support sample weights, a bootstrap sample was generated. Training instances were drawn with replacement according to the current probability distribution w. This forces the weak learner to prioritize previously misclassified points.
- **Weak Learner Training:** A shallow Decision Tree (Max Depth = 3) was trained on the resampled subset.
- **Error Computation:** The learner's weighted error rate (ϵ) was calculated on the original training set:

$$\epsilon = \sum_{i=1}^{N} w_i \cdot \mathbb{I}(y_i \neq G_m(x_i))$$

  where $\mathbb{I}$ is the indicator function and $G_m(x_i)$ is the prediction of the m-th learner.

- **Computing Learner Significance (α):** The influence of the m-th learner was calculated based on its error:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m + 10^{-10}} \right)$$

4.1.4 Adaptive Weight Updating

The sample weights were updated to reflect the difficulty of each instance. Weights for correctly classified samples were decreased, while weights for misclassified samples were increased exponentially:

$$w_i \leftarrow w_i \cdot \exp(-\alpha_m \cdot y_i \cdot G_m(x_i))$$

Following the update, the weights were normalized such that $\sum w_i = 1$ to prepare the distribution for the subsequent iteration.

4.1.5 Final Weighted Ensemble Prediction

The final output is produced via a weighted sum of all M learners. The sign of this sum determines the predicted class, which is then mapped back to the $\{0, 1\}$ format:

$$F(x) = \text{sign} \left(\sum_{m=1}^{M} \alpha_m G_m(x) \right)$$

4.2 Evaluation

4.2.1 Metrics

The following performance metrics were achieved using the optimal hyperparameter configuration—50 estimators, a weak learner max depth of 3, and a PCA variance retention of 0.95—as determined through the validation process:

- **Accuracy:** 0.8587
- **F1-Score:** 0.8738

4.2.2 Confusion Matrix

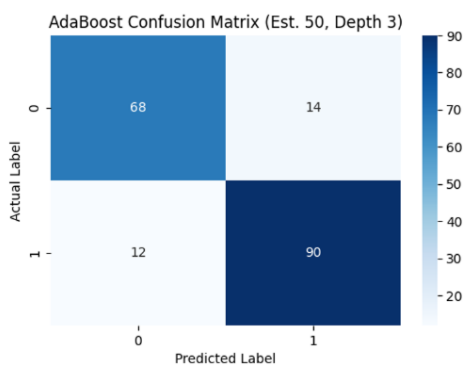

Figure 3: Confusion Matrix for AdaBoost with the Best Hyperparameters

5 Random Forest

While Random Forest is built upon the foundation of Bagging (Bootstrap Aggregating), it introduces a critical modification to the tree-building process. In standard Bagging, each decision tree is grown using a bootstrap sample of the data, but it considers all available features for every split.

Random Forest further decorrelates the trees by injecting randomness into the feature selection process. For each split in every tree, the algorithm only considers a random subset of m features, typically: $m = \sqrt{n}$ where n is the total number of features. By preventing the trees from all choosing the same dominant predictors, the ensemble achieves a more significant reduction in variance.

5.1 Approach

5.1.1 Variance in Bagging

The variance of the average of M identically distributed (but not necessarily independent) random variables, each with variance σ^2 and pairwise correlation ρ, is given by:

$$\text{Var}\left(\frac{1}{M}\sum_{i=1}^{M} X_i\right) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2 \tag{3}$$

By restricting the feature set at each split, Random Forest aggressively lowers ρ, allowing the ensemble to reach a lower total variance than standard Bagging could achieve.

5.1.2 Parallelism

Inherently similar to the Bagged Decision Tree model, the Random Forest algorithm is embarrassingly parallel. Since each tree is constructed independently using a bootstrap sample and a random subset of features, there are no dependencies between individual classifiers. This allows for a linear reduction in training and inference time when distributed across multiple processing units.

In this experiment, we contrasted two implementation approaches:

- **Sequential Training:** Each model is trained in a serial queue, utilizing only a single computational thread.
- **Parallel Training:** Trees are distributed across all available CPU cores, enabling simultaneous model fitting and significantly lower wall-clock time.

Implementation	Description	Training Time (s)
Random Forest (Sequential)	Classifiers are trained one by one on a single CPU core.	1012.3688
Parallel Random Forest	Classifiers are trained concurrently across P available processors.	189.1322

Table 1: Comparison of Sequential vs. Parallel Training Efficiency

In the table above, P represents the number of parallel processing units utilized during the training phase.

5.2 Evaluation

5.2.1 Metrics

The following metrics were achieved using a hyperparameter search—5 Classifiers, a depth of 5, a min samples split of 20 and a PCA variance retention of 0.95—as determined on a cross-validation set

- **Accuracy**: 0.8750
- **F1-Score**: 0.8889

5.2.2 Confusion Matrix

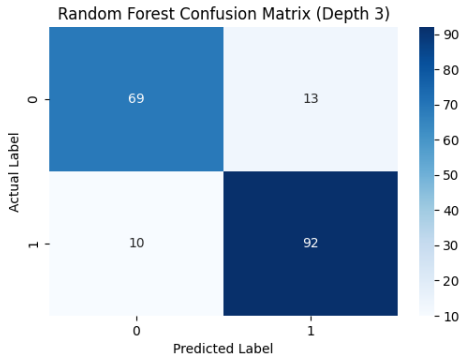

Figure 4: Confusion Matrix for Random Forest with the Best Hyperparameters

6 Analysis and Comparison

6.1 Best-Performing Model

Table 2: Summary of Performance Metrics across Models

Model	Accuracy	F1-Score
Decision Tree	0.8478	0.8692
Bagging Ensemble	0.8641	0.8815
AdaBoost Ensemble	0.8587	0.8738
Random Forest	0.8750	0.8889

6.2 Model Comparisons

The results were extremely close but as our metrics show, The Bagging Ensemble was the best performer (if we don't count the Random Forest bonus) followed by the AdaBoost Ensemble then finally the decision Tree.

6.2.1 Decision Tree VS. Bagging Ensemble

- **Performance Metrics:** The Bagging Ensemble outperformed the single Decision Tree on both metrics (Accuracy: 0.8641 vs. 0.8478, F1-Score: 0.8815 vs. 0.8692). This improvement is the direct result of variance reduction through bootstrap aggregation—averaging 20 independent trees smooths out the instability inherent in a single tree's splits.
- **Confusion Matrices:** The Bagging model reduced the total number of misclassifications compared to the single Decision Tree, achieving a better balance between False Positives and False Negatives. The majority vote mechanism prevents any single tree's erroneous split from dominating the final prediction, leading to more consistent classification across both classes.

6.2.2 Decision Tree VS. AdaBoost Ensemble

- **Performance Metrics:** The AdaBoost model outperformed the single Decision Tree in both general accuracy and the balanced F1-score.
- **Confusion Matrices:** Single decision tree had fewer False Negatives (9) but struggled significantly more with False Positives (19). This model was more "aggressive" in predicting the positive class, while AdaBoost improved the identification of the negative class, reducing False Positives to 14. However, this came at the cost of a slight increase in False Negatives (12). In conclusion, AdaBoost provided a more balanced and conservative classification, which resulted in higher overall accuracy.

AdaBoost successfully leveraged the iterative "weight-correction" process to boost the performance of shallow trees, resulting in a model that generalizes better to unseen test data.

6.3 Misclassification Patterns

Analysis of the confusion matrices reveals a distinct disparity in class-specific errors between the greedy learners and the decorrelated ensembles. Both the single Decision Tree and the Bagging model exhibited a pronounced bias toward False Positives (Actual Class 0 predicted as Class 1), maintaining a misclassification ratio of approximately 1.5:1. This suggests that these models were over-sensitized to dominant clinical features, leading to "noisy" healthy patients being incorrectly flagged as heart-failure risks.

In contrast, the Random Forest and AdaBoost models demonstrated a more robust error distribution, with the misclassification ratio approaching a balanced 1:1. The Random Forest achieved this through feature subsetting, which prevented dominant features from overwhelming the ensemble's logic, while AdaBoost's sequential weighting forced subsequent weak learners to specifically correct the False Positive trends of earlier iterations.